



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 1
по курсу «Компьютерные сети»
«Простейший протокол прикладного уровня»

Студент группы ИУ9-32Б Ивенкова О. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Задание

Целью работы является знакомство с принципами разработки протоколов прикладного уровня и их реализацией на языке Go.

Выполнение лабораторной работы состоит из двух частей.

- Разработать вариант протокола: протокол многократного вычисления значений полинома в различных точках. Протокол должен базироваться на текстовых сообщениях в формате JSON. Результатом разработки протокола должен быть набор типов языка Go, представляющих сообщения, и документация к ним в виде комментариев в исходном тексте.

- Написать на языке Go клиент и сервер, взаимодействующие по разработанному протоколу.

Основные требования к клиенту и серверу:

- полная проверка данных, получаемых из сети (необходимо учитывать, что сообщения могут приходить в неправильном формате и в неправильном порядке, а также могут содержать неправильные данные);

- устойчивость к обрыву соединения;

- возможность одновременного подключения нескольких клиентов к одному серверу;

- сервер должен вести подробный лог всех ошибок, а также других важных событий

- (установка и завершение соединения с клиентом, приём и передача сообщений, и т.п.).

2 Результаты

Исходный код программы представлен в листинге 1–3.

Листинг 1: Реализация файла proto.go

```
1 package proto
2
3 import "encoding/json"
4
5 type Request struct {
6     Command string `json:"command" `
7     Data *json.RawMessage `json:"data" `
```

```

8
9 }
10
11 type Response struct {
12     Status string `json:"status"`
13     Data *json.RawMessage `json:"data"`
14 }
15
16 type Polynomial struct {
17     Degree int `json:"degree"`
18     Coefficients []float64 `json:"coefficients"`
19     X float64 `json:"x"`
20 }

```

Листинг 2: Реализация файла server.go

```

1 package main
2
3 import (
4     "encoding/json"
5     "flag"
6     "fmt"
7     "github.com/mgutz/logxi/v1"
8     "math"
9     "net"
10 )
11
12 import "lab1/src/proto"
13
14 type Client struct {
15     logger log.Logger
16     conn *net.TCPConn
17     enc *json.Encoder
18     polynomials []proto.Polynomial
19 }
20
21 func NewClient(conn *net.TCPConn) *Client {
22     return &Client{
23         logger: log.New(fmt.Sprintf("client %s", conn.RemoteAddr().String()),),
24         conn: conn,
25         enc: json.NewEncoder(conn),
26         polynomials: []proto.Polynomial{},
27     }
28 }
29
30 func (client *Client) serve() {

```

```

31     defer client.conn.Close()
32     decoder := json.NewDecoder(client.conn)
33     for {
34         var req proto.Request
35         if err := decoder.Decode(&req); err != nil {
36             client.logger.Error("cannot decode message", "
reason", err)
37             break
38         } else {
39             client.logger.Info("received command", "command"
, req.Command)
40             if client.handleRequest(&req) {
41                 client.logger.Info("shutting down
connection")
42                 break
43             }
44         }
45     }
46 }
47
48 func (client *Client) handleRequest(req *proto.Request) bool {
49     switch req.Command {
50     case "quit":
51         client.respond("ok", nil)
52         return true
53     case "add":
54         errorMsg := ""
55         if req.Data == nil {
56             errorMsg = "data field is absent"
57         } else {
58             var polynomial proto.Polynomial
59             if err := json.Unmarshal(*req.Data, &polynomial)
; err != nil {
60                 errorMsg = "malformed data field"
61             } else {
62                 client.logger.Info("adding polynomial",
"polynomial", polynomial)
63                 client.polynomials = append(client.
polynomials, polynomial)
64             }
65         }
66         if errorMsg == "" {
67             client.respond("ok", nil)
68         } else {
69             client.logger.Error("addition failed", "reason",
errorMsg)

```

```

70             client.respond("failed", errorMsg)
71         }
72     case "avg":
73         if len(client.polynomials) == 0 {
74             client.logger.Error("calculation failed", "
reason", "no polynomials added")
75             client.respond("failed", "no polynomials added")
76         } else {
77             lastPolynomial := client.polynomials[len(client.
polynomials)-1]
78             result := evaluatePolynomial(lastPolynomial)
79             client.respond("result", result)
80         }
81     default:
82         client.logger.Error("unknown command")
83         client.respond("failed", "unknown command")
84     }
85     return false
86 }
87
88 func evaluatePolynomial(polynomial proto.Polynomial) float64 {
89     result := 0.0
90     for i := 0; i <= polynomial.Degree; i++ {
91         result += polynomial.Coefficients[i] * math.Pow(
polynomial.X, float64(i))
92     }
93     return result
94 }
95
96 func (client *Client) respond(status string, data interface{}) {
97     var raw json.RawMessage
98     raw, _ = json.Marshal(data)
99     client.enc.Encode(&proto.Response{status, &raw})
100 }
101
102 func main() {
103     var addrStr string
104     flag.StringVar(&addrStr, "addr", "185.102.139.168:5451", "
specify ip address and port")
105     flag.Parse()
106
107     if addr, err := net.ResolveTCPAddr("tcp", addrStr); err != nil {
108         log.Error("address resolution failed", "address",
addrStr)
109     } else {

```

```

110         log.Info("resolved TCP address", "address", addr.String
111         ())
112         if listener, err := net.ListenTCP("tcp", addr); err !=
113         nil {
114             log.Error("listening failed", "reason", err)
115         } else {
116             for {
117                 if conn, err := listener.AcceptTCP();
118                 err != nil {
119                     log.Error("cannot accept
120                     connection", "reason", err)
121                 } else {
122                     log.Info("accepted connection",
123                     "address", conn.RemoteAddr().String())
124                     go NewClient(conn).serve()
125                 }
126             }
127         }
128     }
129 }

```

Листинг 3: Реализация файла client.go

```

1 package main
2
3 import (
4     "encoding/json"
5     "flag"
6     "fmt"
7     "github.com/skorobogatov/input"
8     "net"
9 )
10
11 import "lab1/src/proto"
12
13 func interact(conn *net.TCPConn) {
14     defer conn.Close()
15     encoder, decoder := json.NewEncoder(conn), json.NewDecoder(conn)
16     for {
17         fmt.Printf("command = ")
18         command := input.Gets()
19
20         switch command {
21         case "quit":
22             send_request(encoder, "quit", nil)
23             return

```

```

24         case "add":
25             fmt.Printf("degree = ")
26             var degree int
27             fmt.Scan(&degree)
28
29             coefficients := make([] float64, degree + 1)
30             fmt.Printf("coefficients = ")
31             for i := 0; i <= degree; i++ {
32                 fmt.Scan(&coefficients[i])
33             }
34
35             fmt.Printf("point = ")
36             var point float64
37             fmt.Scan(&point)
38
39             polynomial := proto.Polynomial{
40                 Degree: int(degree),
41                 Coefficients: coefficients,
42                 X: point,
43             }
44             send_request(encoder, "add", &polynomial)
45         case "avg":
46             send_request(encoder, "avg", nil)
47     default:
48         fmt.Printf("error: unknown command\n")
49         continue
50     }
51
52     var resp proto.Response
53     if err := decoder.Decode(&resp); err != nil {
54         fmt.Printf("error: %v\n", err)
55         break
56     }
57
58     switch resp.Status {
59     case "ok":
60         fmt.Printf("ok\n")
61     case "failed":
62         if resp.Data == nil {
63             fmt.Printf("error: data field is absent
64 in response\n")
65         } else {
66             var errorMsg string
67             if err := json.Unmarshal(*resp.Data, &
errorMsg); err != nil {

```

```

67                                     fmt.Printf("error: malformed
data field in response\n")
68                                     } else {
69                                     fmt.Printf("failed: %s\n",
errorMsg)
70                                     }
71                                     }
72     case "result":
73         if resp.Data == nil {
74             fmt.Printf("error: data field is absent
in response\n")
75         } else {
76             var result float64
77             if err := json.Unmarshal(*resp.Data, &
result); err != nil {
78                 fmt.Printf("error: malformed
data field in response\n")
79             } else {
80                 fmt.Printf("Polynomial result: %
f\n", result)
81             }
82         }
83     default:
84         fmt.Printf("error: server reports unknown status
%q\n", resp.Status)
85     }
86 }
87 }
88
89 func send_request(encoder *json.Encoder, command string, data interface
{}) {
90     var raw json.RawMessage
91     raw, _ = json.Marshal(data)
92     encoder.Encode(&proto.Request{command, &raw})
93 }
94
95 func main() {
96     var addrStr string
97     flag.StringVar(&addrStr, "addr", "185.102.139.168:5451", "
specify ip address and port")
98     flag.Parse()
99
100    if addr, err := net.ResolveTCPAddr("tcp", addrStr); err != nil {
101        fmt.Printf("error: %v\n", err)
102    } else if conn, err := net.DialTCP("tcp", nil, addr); err != nil
{

```



```

103         fmt.Printf("error: %v\n", err)
104     } else {
105         interact(conn)
106     }
107 }

```

Результат запуска представлен на рисунках 1– 2.

```

root@net3:~/Ivenkova/lab1# go build -o bin/server src/server/server.go
root@net3:~/Ivenkova/lab1# go build -o bin/client src/client/client.go
root@net3:~/Ivenkova/lab1# ./bin/server
17:16:39.661165 INF ~ resolved TCP address
address: 185.102.139.168:5450
17:16:49.343432 INF ~ accepted connection
address: 185.102.139.168:39558
17:17:20.300283 INF client 185.102.139.168:39558 received command
command: add
17:17:20.301728 INF client 185.102.139.168:39558 adding polynomial
polynomial: map[coefficients:[4 5 3] degree:2 x:2]
17:17:24.457630 INF client 185.102.139.168:39558 received command
command: avg
17:17:43.644228 INF client 185.102.139.168:39558 received command
command: quit
17:17:43.644553 INF client 185.102.139.168:39558 shutting down connection

```

Рис. 1 — Результат со стороны сервера

```

root@net3:~/Ivenkova/lab1# ./bin/client
command = add
degree = 2
coefficients = 4 5 3
point = 2
ok
command = avg
Polynomial result: 26.000000
command = quit

```

Рис. 2 — Результат со стороны клиента